Trabajo Práctico Integrador (UVA 1 y UVA 2): Arquitectura del Sistema de Control Sky-Delivery S.A. 1.

Integrantes:

Facendini, Nicolas - 1140673

Felice, Lucas - 1141923

Leiva, Maria Victoria - 1157085

Palma, Mirko Rodrigo - 1201583

Segovia De Leon, Valentina - 1167893

Vallejos Vitalevi, Alejandro David - 1146637

Villalba, Ignacio - 1215744

Zaldivar Suarez, Ramiro - 1207185

1. Contexto de Negocio: Infraestructura de Logística Autónoma

La corporación Sky-Delivery S.A. se encuentra en una fase crítica de despliegue tecnológico. Es imperativo que la arquitectura de software que soporte su flota de drones de reparto sea robusta, escalable y esté diseñada bajo el rigor del Paradigma Orientado a Objetos (POO) . No se admitirán soluciones basadas en lógica procedimental; se exige un diseño que mitigue redundancia mediante el encapsulamiento y un desacoplamiento efectivo entre componentes.El sistema debe coordinar de manera precisa las siguientes entidades:

- Drones: Unidades autónomas que gestionan internamente niveles de energía y telemetría (coordenadas GPS).
- Paquetes: Entidades con dimensiones físicas y peso que condicionan la autonomía del transporte.
- Centros de Despacho: Nodos logísticos responsables de la orquestación y asignación de rutas.Como arquitectos, su misión es garantizar que el diseño soporte la complejidad operativa de una flota de alta disponibilidad, asegurando que la transición entre el modelo teórico y la ejecución en Máquina Virtual sea impecable.

2. Desafío de Modelado UML (UVA 1)

El modelado no es un freno a la programación, sino el "GPS" que ordena la lógica sistémica. Se requiere una representación fiel de la estructura estática.

2.1 Identificación de Componentes y los 3 Pilares

Identifique cuatro clases nucleares para el escenario planteado. Para cada clase, es obligatorio documentar:

- 1. ESTADO (Qué sabe): Defina al menos 3 atributos específicos querepresenten su situación actual.
- 2. COMPORTAMIENTO (Qué hace): Defina al menos 3 métodos o acciones que la clase puede ejecutar.
- 3. IDENTIDAD (Cuál es cuál): Determine el mecanismo técnico que permite distinguir una instancia de otra, garantizando que cada objeto sea único en el ecosistema de ejecución

Respuesta:

Clase 1: Drone

ESTADO (Qué sabe):

- `idDron: int`
- `estado: enu`
- `capacidadCargaMax`: double

COMPORTAMIENTO (Qué hace):

- `iniciarVuelo():void`
- `autorizarVuelo():bool`
- `volverABase(): void`

IDENTIDAD: El `idDrone` es único, como una patente. Nunca hay dos drones con el mismo ID.

Clase 2: BateriaInterna

ESTADO:

- `idBateriaInterna: int`
- `capacidadMaxima` : double

- `temperatura`: double

COMPORTAMIENTO:

- `recargar()`: void
- `volverABase(): void`
- `consumirEnergia(): void`

IDENTIDAD: El `idBateria`. Además, como hay composición 1:1 con Drone, también se identifica por el drone al que pertenece.

Clase 3: Paquete

ESTADO:

- `idPaquete: int`
- `peso`:double
- `dimensiones` :string

COMPORTAMIENTO:

- `calcularVolumen():double`
- `obtenerPeso():double`
- ActualizarEstadoEntrega(): void

IDENTIDAD: El `idPaquete` funciona como código de tracking. Dos paquetes con mismo peso y destino siguen siendo objetos distintos.

Clase 4: CentroDespacho

ESTADO:

- `idCentro: int`
- `ubicacion` :String
- `nombre` :string

COMPORTAMIENTO:

- `registrarDrone(d Drone): void`
- `despacharPaquete(p Paquete)` : bool
- `asignarRuta(): void`

IDENTIDAD: El `idCentro` distingue cada centro. También se puede complementar con la ubicación geográfica que es única.

2.2 Relaciones y Preservación de la Integridad

Documente las conexiones entre clases, determinando con precisión la cardinalidad y la direccionalidad de los mensajes.

- Instrucción obligatoria: Identifique al menos una relación de composición (por ejemplo, entre el Drone y su BateríaInterna). Justifique Por qué la existencia de la parte está supeditada a la del todo en este contexto de negocio.

Respuesta

Drone → BateriaInterna - COMPOSICIÓN (1:1)

- El drone crea su batería y es el único dueño. El mensaje va del Drone hacia la BateriaInterna (el drone le pide carga, le dice que gaste energía, etc.). La batería nunca le habla al centro ni al paquete, solo responde al drone.

CentroDespacho → Drone - AGREGACIÓN (1 a muchos)

- Un centro tiene 0 o muchos drones. El centro les manda mensajes: "despega", "tomá este paquete", "reporta estado". Pero el drone puede existir sin centro (por ejemplo, en mantenimiento externo o siendo trasladado).

CentroDespacho → Paquete - AGREGACIÓN (1 a muchos)

- Un centro tiene 0 o muchos paquetes en cola. El centro los gestiona y despacha. Pero el paquete existe antes de llegar al centro (alguien lo armó, lo pesó, le puso dirección) y sigue existiendo después de entregado.

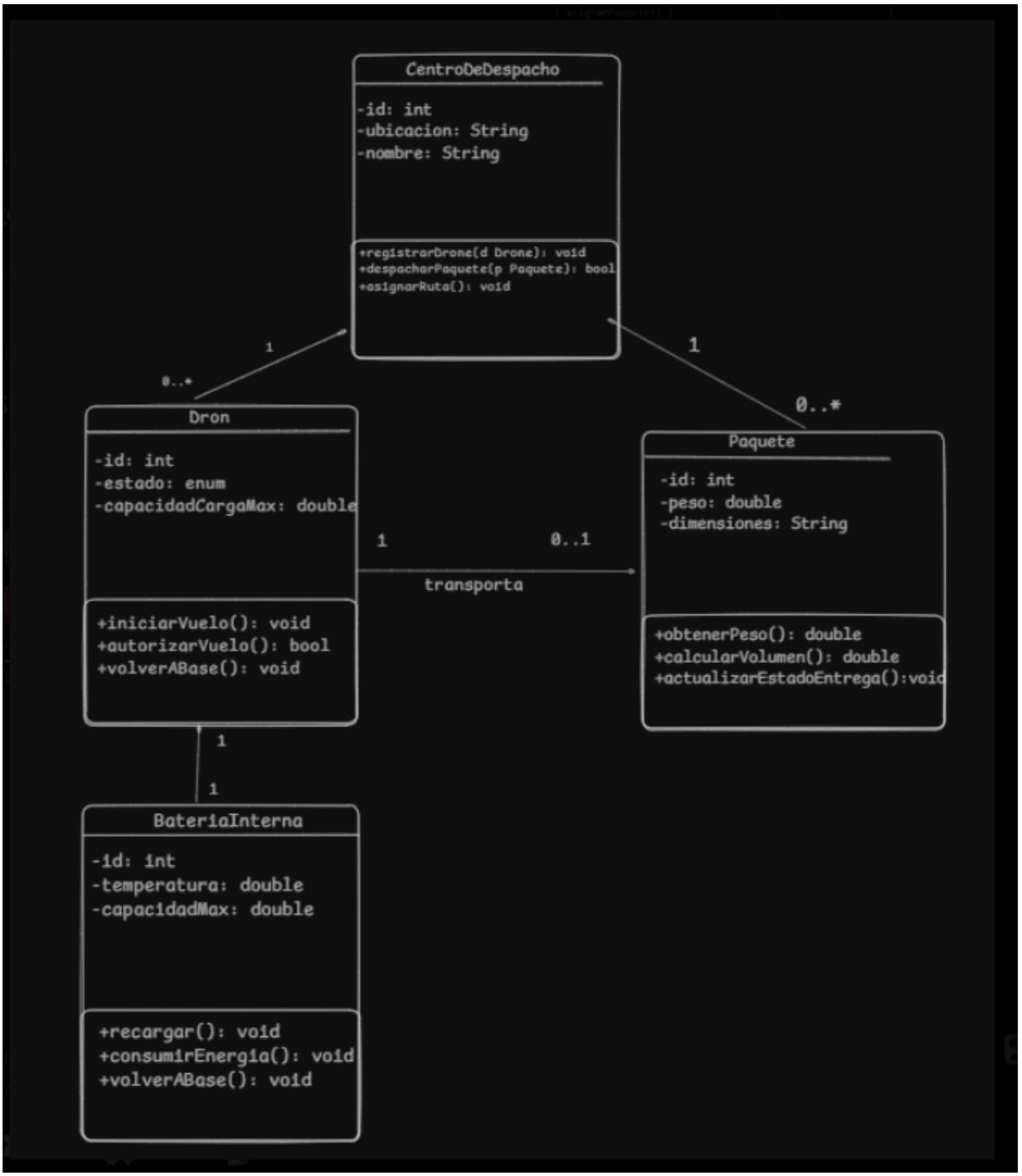
Drone → Paquete - ASOCIACIÓN (1 a 0..1)

- Un drone puede llevar 0 o 1 paquete a la vez. El drone conoce al paquete que transporta, pero el paquete no necesita saber qué drone lo lleva. La relación es temporal: dura lo que dura el viaje.

2.3 El Diagrama de Clases: Del "Molde" a la "Realidad"

Construya un Diagrama de Clases UML detallado. En su entrega, debe distinguir explícitamente entre el "Molde" (La Clase) como definición estática y"LaRealidad" (Los Objetos) como instancias concretas que interactúan. Utilice la notación estándar para representar la visibilidad y las firmas de los métodos.

Respuesta


CentroDeDespacho
-id: int
-ubicacion: String
-nombre: String
+registrarDrone(d Drone): void
+despacharPaquete(p Paquete): bool
+asignarRuta(): void
1
0..*
1
0..*
Dron
-id: int
-estado: enum
-capacidadCargaMax: double
+iniciarVuelo(): void
+autorizarVuelo(): bool
+volverABase(): void
1
0..1
transporta
Paquete
-id: int
-peso: double
-dimensiones: String
+obtenerPeso(): double
+calcularVolumen(): double
+actualizarEstadoEntrega():void
1
1
BateriaInterna
-id: int
-temperatura: double
-capacidadMax: double
+recargar(): void
+consumirEnergia(): void
+volverABase(): void

3. Desafíos Técnicos de Arquitectura y Memoria (UVA 2)

3.1 Gestión en la Máquina Virtual (VM)

- ### Segmentación de Memoria

Las variables de referencia de un objeto `Drone` se almacenan en el Stack porque forman parte de la ejecución de los métodos y contienen únicamente la dirección de memoria del objeto.
La instancia del objeto se almacena en el Heap, ya que allí se guardan los objetos creados dinámicamente.

Cuando la variable pierde alcance, se elimina del Stack. Si no existen más referencias al objeto en el Heap, este queda disponible para ser eliminado por el Garbage Collector.

- ### Ciclo de Vida en la VM

El proceso comienza con el código fuente en archivos `.java`, que es compilado por `javac` y transformado en bytecode (`.class`).
Luego, la JVM carga las clases mediante el ClassLoader y verifica que el bytecode sea válido.

Finalmente, la JVM ejecuta el programa, interpretando el bytecode o compilándolo mediante JIT. Durante la ejecución se crean objetos en memoria y el Garbage Collector elimina los que ya no se utilizan.

3.2 Tipos de Datos y Wrappers

El método `consumirEnergia()` debe ser de tipo `void` porque solo realiza una acción sobre el objeto y no necesita devolver ningún valor.

3.3 Estructuras de Control y Manejo de Arreglos

`Método autorizarVuelo()`

```
public boolean autorizarVuelo()
```

```
    si no hay paquete asignado
        retornar false

    obtener peso del paquete
    obtener nivel de batería

    si la batería es suficiente para ese peso
        retornar true
    sino
        retornar false
```

Este método valida si el drone tiene un paquete asignado y si el nivel de batería es suficiente para transportarlo. Si ambas condiciones se cumplen, retorna verdadero; en caso contrario, retorna falso.

Instanciación de una flota de 50 drones

Primero se declara el arreglo:

```
Drone[ ] flota = new Drone[50];
```

Esta instrucción solo crea un arreglo de referencias, donde todas las posiciones se inicializan en `null`.

Luego, es necesario instanciar cada objeto individualmente:

```
for (int i = 0; i < flota.length; i++) {
   flota[i] = new Drone();
}
```

4. Integración Dinámica: Diagrama de Secuencia

4.1 Escenario de Interacción Crítica Modele el siguiente evento:

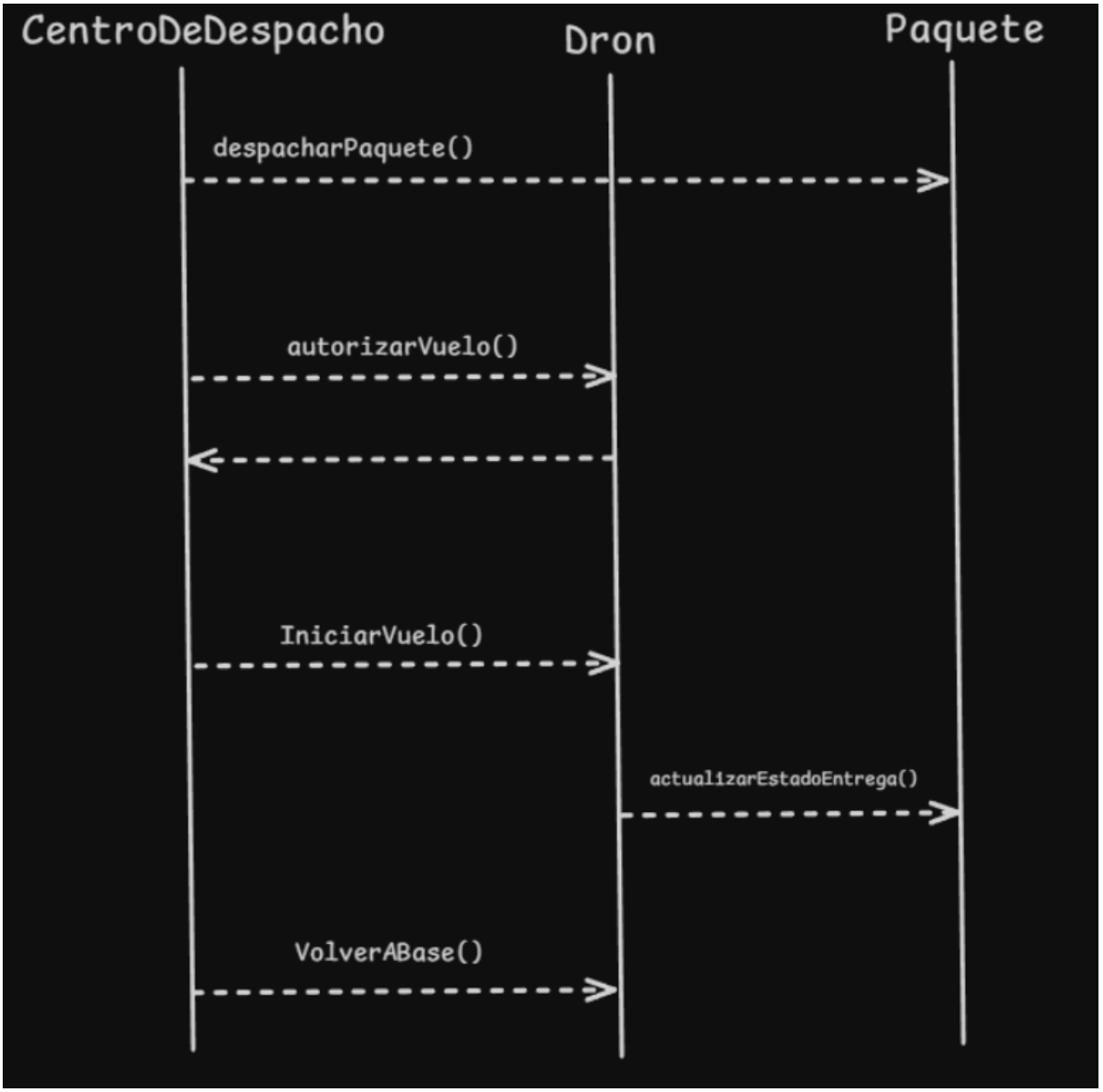
"Un Centro de Despacho asigna un Paquete a un Drone; este válida su autonomía, luego de la confirmación, inicia el protocolo de despegue" .

4.2 Rigor en la Mensajería El Diagrama de Secuencia:

Debe respetar estrictamente el orden temporal y la interacción entre Centro de Despacho, Paquete y Drone.

Se exige la inclusión de:

- Líneas de Vida y Activaciones : Para representar el tiempo de control de cada objeto.
- Mensajes de Retorno : Para confirmar la finalización de las validaciones internas.
- Auto-mensajes : En caso de que el Drone realice validaciones internas de telemetría.


CentroDeDespacho
Dron
Paquete
despacharPaquete()
autorizarVuelo()
IniciarVuelo()
actualizarEstadoEntrega()
VolverABase()